

CP31

- <https://codeforces.com/profile/kintsugi-programmer>
- <https://www.tle-eliminators.com/cp-sheet>

Table of Contents

- [CP31](#)
 - [Table of Contents](#)
- [R800](#)
 - [1 Halloumi Boxes](#)
 - [2 Line Trip](#)
 - [3 Cover in Water](#)
 - [4 Game with Integers](#)
 - [5 Jagged Swaps](#)
 - [6 Doremy's Paint 3](#)
 - [7 Don't Try to Count](#)
 - [8 How Much Does Daytona Cost?](#)
 - [10 Target Practice](#)
- [TipsCollectedFromExperiences](#)

R800

1 Halloumi Boxes

- <https://codeforces.com/problemset/problem/1903/A>
- Analysis
 - n boxes/array a1 a2 ... an
 - He wants to sort them in non-decreasing order based on their number
 - however, his machine works in a strange way. It can only reverse any subarray of boxes with length **at most k**
 - subarrsize≤k
 - Find if it's possible to sort the boxes using any number of reverses.
 - So, if K≥2 ,Machine's sort is 100%possible at **any number of reverses ANY_TIMES**
 - eg:
 - 6421
 - 6421
 - 6412
 - 6142
 - 1642
 - 1624
 - 1264
 - 1246
 - sorting done :0
 - at k=1, no sorting is possible

- Approach
 - Passing Condition where return YES
 - $K \geq 2$
 - or given array is already sorted
 - else, return False

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    // at extreme proof case use :
    // long long t;
    int t;
    cin>>t;
    while(t--){
        int n,k;
        cin>>n>>k;
        vector<int> v(n);
        int i=0;
        while(i<n){
            cin>>v[i];
            i++;
        }
        vector<int> v2=v;
        sort(v2.begin(), v2.end());
        if ((v2==v)or(k>=2)){
            cout<<"YES\n";
        } else {
            cout<<"NO\n";
        }
    }
    return 0;
}
// tc O(nlogn)
// sc O(n)3
```

2 Line Trip

- <https://codeforces.com/problemset/problem/1901/A>
- Analysis
 - location&road on number line
 - 0 , a1 , a2 , ... , x
 - Round Trip
 - a1,a2 ... Gas station for Tanki Full
 - 0 start point
 - if stop at non-station location= gameover
 - In this ques, we have to find the capacity of gas tank car should take it for journey
 - now the max capacity of gas tank in any journey = max distance of any 2 gas stations throughout journey
 - throughtout journey means a round trip

- so, after lastGasStation, car will go to x(dest), and find lastGasStation first in return journey
- so, that distance is (lastGasStation - x)*2
- Approach
 - C1= cal. firstGasStation-0
 - C2= max(allDistances(cal. dist b/w eachGasStation))
 - C3= cal. (lastGasStation - x)*2
 - return max(C1,C2,C3)

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    long long t;
    cin>>t;
    while(t--){
        long long n,x;
        cin>>n>>x;
        long long i=1;
        long long smax=0;
        vector<long long> v1(n+1,0);
        v1[0]=0;
        while(i<=n){
            cin>>v1[i];
            // cout<<v1[i];
            long long buff=smax;
            smax=max(buff,(v1[i]-v1[i-1]));
            i++;
        }
        long long smax2=(x-v1[n])*2;
        cout<< max(smax,smax2)<<endl;
        // cout<< smax<<" "<<v1[n-1]<<" "<<smax2<<endl;

    }return 0;
}
```

3 Cover in Water

- <https://codeforces.com/problemset/problem/1900/A>
- Analysis
 - Filip has a row of cells, some of which are blocked, and some are empty.
 - He wants all empty cells to have water in them.
 - He has two actions at his disposal
 - 1. place water in an empty cell. **FINITE_TIMES**
 - 2. remove water from a cell and place it in any other empty cell. **ANY_TIMES**
 - autoOperation
 - if at some moment cell i ($2 \leq i \leq n-1$) is empty and both cells i-1 and i+1 contains water, then it becomes filled with water. **ANY_TIMES**
 - C1: if no. of Consecutive dots(emptyBoxes) ≥ 3

- then we only need 1 operation only 2TIMES as we could fill at corner of 3 boxes and middle box will autoOperationAutoFillWater and we can transfer that water to other cells ANY_TIMES 2 operation, and regenerate autoOperationAutoFillWater
 - C2: else we need to fill all boxes by 1 operation only as here autoOperationAutoFillWater fails
- Approach
 - count no. of dots(emptyBoxes)
 - count no. of Consecutive dots(emptyBoxes)
 - if no. of Consecutive dots(emptyBoxes) >= 3
 - return 2
 - else
 - return no. of dots(emptyBoxes)
- AnotherApproach
 - similar
 - if (s[i]=="." && i+1<n && s[i+1]=="." && i+2<n && s[i+2]==".")
 - return 2
 - else
 - return no. of dots(emptyBoxes)

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    long long t;
    cin>>t;
    while (t--){
        long long n;
        cin>>n;

        string s;
        cin>>s;

        int sum=0;
        int i=0;
        int dot=0;

        while(i<s.size()){
            if (s[i]=='.' & sum!=3)
            {
                sum++;
                dot++;
            }
            else if (s[i]=='#' & sum!=3)
            {
                sum=0;
            }
            i++;
        }

        if (sum>=3)
```

```

    {
        cout<<2<<endl;
    }
    else
    {
        cout<<dot<<endl;
    }

}
return 0;

}

```

4 Game with Integers

- <https://codeforces.com/problemset/problem/1899/A>
- Analysis
 - Vanya and Vova are playing a game. Players are given an integer n . On their turn, the player can add 1 to the current integer or subtract 1
 - The players take turns; Vanya starts. If after Vanya's move the integer is divisible by 3, then he wins. If 10 moves have passed and Vanya has not won, then Vova wins.
 - if both players play optimally
 - then in each of player turn he/she will try move to make other one lose
 - NOW
 - $n \% 3 == 0$
 - if True, divisible before vanya move
 - even vanya could +1/ -1
 - it will not be divisible by 3
 - & vova will cancel the effect -1/ +1 of vanya till 10 rounds
 - ultimate vova win
 - if False, not divisible before vanya move
 - vanya could +1/ -1
 - it will be divisible by 3 after vanya move
 - & vova will do something
 - & vova will cancel the effect -1/ +1 of vova
 - and still it will be divisible by 3 after vanya move till 10 rounds
 - ultimate vanya win
 - numbers
 - 0 DIV
 - 1 (-1=0)
 - 2 (+1=3)
 - 3 DIV
 - 4 (-1)
 - 5 (+1)
 - 6 DIV
 - 7

- 8
- 9 DIV
- Eg: 6
 - $\Rightarrow 7 \Rightarrow 8 \Rightarrow 9 \Rightarrow \dots$ vanya win
 - $\Rightarrow 7 \Rightarrow 6 \Rightarrow 7 \Rightarrow 6 \dots$ vova win if played optimally
- Approach
 - if $n\%3 == 0$
 - vova win, return Second
 - else if $n\%3 != 0$
 - vanya win, return Second

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    long long t=0;
    cin>>t;
    while(t--){
        long long n=0;
        cin>>n;
        if (n%3==0){cout<<"Second\n";}
        else{cout<<"First\n";}
    }
    return 0;
}
```

5 Jagged Swaps

- <https://codeforces.com/problemset/problem/1896/A>
- Analysis
 - here, permutation is an array
 - of unique elements
 - if array of n integers, then integer exists of all $1, \dots, n$
 - we want to check if we could sort the permutation with special operation **ANY_TIMES**
 - special operation
 - if $a[i-1] < a[i] > a[i+1]$
 - then swap $a[i], a[i+1]$
- Approach
 - if this is permutation
 - if permutation is already sorted
 - return "YES"
 - else
 - try sort by special operation one time
 - if sorted
 - return "YES"
 - else
 - return "NO"

- else
 - return "NO"
 - TTYL

6 Doremy's Paint 3

- <https://codeforces.com/problemset/problem/1890/A>
- Analysis
 - array is good for this ques as
 - there exists a k such that $b_1+b_2=b_2+b_3=\dots=b_{n-1}+b_n=k$
- Approach
 - and at case of array of 2, 1 elements
 - return "Yes"
 - else
 - `int k = a[1]-a[0];`
 - run a loop
 - `if ( i>-1 && i+1<n && a[i]+a[i+1]==k)`
 - or reverse also k [TTYL]
 - return "Yes"
 - else
 - return "No"

7 Don't Try to Count

- <https://codeforces.com/problemset/problem/1881/A>
- Analysis
 - string x, len n
 - string s, len m
 - $n*m \leq 25$
 - 1, 25
 - 5, 5
 - 25, 1
 - operation `ANY_TIMES`
 - `if x= "abc"`
 - `return x+x "abcabc"`
 - Find
 - min. no of operation by which
 - s is substring of x
- Approach
 - input t testcases
 - each test cases
 - input n,m
 - Wrong, as babb,bbb ,its not -1
 - check if x is substring of s+s

- if no
 - then x ,even mul by infinite can't be superset or in any combination of s
 - and we took s+s as maybe x="mara", s="rama"
 - return -1
 - if yes
 - then its posible
- counter=0
- while counter<=5
 - concatnate till s is substring of x
 - counter++
- return counter

8 How Much Does Daytona Cost?

- <https://codeforces.com/problemset/problem/1878/A>
- Analysis
 - array a
 - size n
 - int k
 - Find?
 - subsegment of a where k is most common element
 - One sec,
 - if number exists
 - if array size is 2 or 1
 - or if anywhere $a[i]=a[i+1]=k$
 - or if its>2
 - if in array bw that 1st occur and last occur ,that number the most occur
 - return yes if any satisfy, else no :0
- Approach
 - FREAKING, the the limits, loopholes are hidden
 - functions returns>if else with breaks
 - if number exists
 - then its largest at subarray len = 1

10 Target Practice

- Analysis
 - given board
 - 10x10
 - each ring deeper, more points, outermost is 1, innermost is 5
 - person shot "X" in grid of 10x10 "."
 - Find?
 - Scores
 - technically


```
0123456789
```

```
1111111111 0
1222222221 1
1233333321 2
1234444321 3
1234554321 4
1234554321 5
1234444321 6
1233333321 7
1222222221 8
1111111111 9
```

```
0 1234 5678 9
```

```
1 1111 1111 1 0
```

```
1 2222 2222 1 1
1 2333 3332 1 2
1 2344 4432 1 3
1 2345 5432 1 4
```

```
1 2345 5432 1 5
1 2344 4432 1 6
1 2333 3332 1 7
1 2222 2222 1 8
```

```
1 1111 1111 1 9
```

- condition of scoring

- 1

- 0,0 to 9,0 L 00 10 20 30 40 50 60 70 80 90
- 0,9 to 9,9 R 09 19 29 39 49 59 69 79 89 99
-
- 0,0 to 0,9 T 00 01 02 03 04 05 06 07 08 09
- 9,0 to 9,9 B 90 91 92 93 94 95 96 97 98 99
- relation
 - $i = 0$ or 9
 - $j = 0$ or 9
 - $1 = 0+1 == 10-9$

- 2

- 1,1 to 8,1 L 11 21 31 41 51 61 71 81
- 1,8 to 8,8 R 18 28 38 48 58 68 78 88
-
- 1,1 to 1,8 T 11 12 13 14 15 16 17 18
- 8,1 to 8,8 B 81 82 83 84 85 86 87 88
- relation
 - $i, j = 1$ or 8
 - $1 = 1+1 == 10-8$

- 5
 - 4,4
 - 4,5
 - 5,4
 - 5,5
- this not 2D Array
- this is char incoming
- Approach

TipsCollectedFromExperiences

- when check TC& SC of program, don't consider TestCasesLoop&Spaces in counting
- 1sec = 10^8 Operations
- if 1sec = totalTests
 - operationsPerTestCase = $10^8 / \text{totalTestCases}$
 - if totalTestCases = 100
 - operationsPerTestCase = 10^6 operations
 - $O(n^3)$ is UpperLimit of the question's code
 - as $O(n^3) = O(100^3) = 10^6 === \text{operationsPerTestCase}$
- always think of extra testcases
- and in cp submission , you can see testcases in ID :0
- FREAKING, the the limits, loopholes are hidden
- functions returns>if else with breaks
- put this at 1st line of main() code, to fix bug of compiler at running test cases, not interactive program

```
ios::sync_with_stdio(0);
cin.tie(0);
```

- templates

```
// template miniTests int1 int2 vectorArrayInt1
#include<bits/stdc++.h>
using namespace std;
void mainGame(int x1, int x2, vector<int> v1){
    // code here

}

void eachMiniTest(){
    int x1=0, x2=0;// factor1 factor2
```

```

        vector<int> v1;
        cin>>x1>>x2;
        for (int i=0; i<x1; i++ ) cin>>v1[i];
        mainGame(x1,x2,v1);

    }

    int main(){
        ios::sync_with_stdio(0);
        cin.tie(0);
        long long t; //mini test cases
        cin>>t;
        while(t--){
            eachMiniTest();
        }

        return 0;
    }

```

```

// template miniTests int1 int2 string
#include<bits/stdc++.h>
using namespace std;
void mainGame(int x1, int x2, string s){
    // code here

}

void eachMiniTest(){
    int x1=0, x2=0;// factor1 factor2
    string s;
    cin>>x1>>x2>>s;
    mainGame(x1,x2,s);

}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    long long t; //mini test cases
    cin>>t;
    while(t--){
        eachMiniTest();
    }

    return 0;
}

```

- Vectors CPP STL

```
vector<int> v1(n); // create ,with n elements mandatory
cin>>v1[0]; // insert
vector<int> v2 = v1; // copy
sort(v2.begin(),v2.end()); // sorting in stl, asc
bool compare= (v1==v2); // compare
```

```
#include<vector>
#include<iostream>
#include<algorithm>
```

- use `long long` instead of `int` for bigger stuff
- max/min : `max(var1 ,var2),min(var1, var2)`
 - make sure var1,var2 has SAME DATATYPE
 - inbuilt